

Les fonctions conditionnelles et logiques (1/11)

Les instructions conditionnelles permettent de provoquer des actions en fonction de la réalisation de conditions. Le principe d'une instruction conditionnelle est le suivant :

SI condition vraie **ALORS** action1 **SINON** action2

Les instructions conditionnelles servent à sélectionner des observations et à créer de nouvelles bases de données et de nouvelles variables en fonction de la valeur d'autres variables.

Il existe différents types d'instructions conditionnelles :

- L'instruction conditionnelle « **IF... THEN... ; ELSE... ;** »
- L'instruction conditionnelle « **SELECT...END ; OTHERWISE...;** »
- L'instruction conditionnelle « **WHERE** »

Opérateurs de comparaison

- **=** ou **EQ** : égal à
- **^=** ou **NE** : différent
- **>** ou **GT** : plus grand que
- **<** ou **LT** : plus petit que
- **>=** ou **GE** : plus grand ou égal
- **<=** ou **LE** : plus petit ou égal
- **in** (*liste*) : présent dans la liste

Opérateurs logiques

- **AND** : et
- **OR** : ou
- **NOT** : négation

Les fonctions conditionnelles et logiques (2/11)

Les différents types de conditions

Les instructions conditionnelles ont plusieurs applications :

- **Comparer le contenu d'une variable et d'une constante**

Ex : *IF age < 35;*

Condition : l'âge doit être inférieur à 35 ans

- **Effectuer une comparaison entre le contenu d'une variable et le contenu d'une autre variable**

Ex : *IF hagemar > fagemar;*

Condition : l'âge au mariage de l'homme (hagemar) doit être supérieur à l'âge au mariage de la femme (fagemar)

- **Tester le contenu d'une variable par rapport au résultat d'un calcul**

Ex : *IF salaire < (2500+321);*

Condition : le salaire doit être inférieur à la somme indiquée (2821)

- **Tester le contenu d'une variable par rapport à un ensemble de valeurs**

Ex : *IF age < 35 AND age > 15;*

Condition : l'âge doit être à la fois inférieur à 35 et supérieur à 15 ans

- **Sélectionner des unités ou effectuer un test en utilisant plusieurs variables simultanément avec des opérateurs logiques**

Ex : *IF sexe='H' AND age > 12;*

Condition : l'individu doit être un homme ET être âgé de plus de 12

Les fonctions conditionnelles et logiques (3/11)

L'instruction conditionnelle *IF ... THEN ... ; ELSE ... ;*

L'instruction *IF* est une instruction de l'étape DATA.

Elle permet de tester une condition sur l'observation en cours de traitement : SAS vérifie si la condition est vérifiée pour la 1ère observation, si oui il la conserve, sinon il passe à l'observation suivante (etc.)

Syntaxe : IF condition THEN instruction1 ; ELSE instruction2 ;

Principe d'écriture de la procédure : si (IF) la condition est vérifiée alors (THEN) l'instruction1 sera exécutée, sinon (ELSE) c'est l'instruction2 qui le sera.

L'instruction conditionnelle peut servir à créer une nouvelle variable en fonction des valeurs de variables existantes. L'instruction s'inscrit dans une étape DATA.

Exemple : on souhaite calculer la variable "cotis" qui contiendra le montant de la cotisation due par l'enfant pour son adhésion à un club sportif. Ce montant vaut 40 pour les moins de 10 ans, 60 pour les enfants de 10 à moins de 12 ans, et 80 pour les autres.

```
DATA club2 ;  
  set club ;  
  IF age <10 THEN cotis=40 ;  
  ELSE IF age >=10 and age <12 then cotis=60 ;  
  ELSE cotis=80 ;  
  run ;
```

Les fonctions conditionnelles et logiques (4/11)

L'instruction conditionnelle peut servir à filtrer des observations, pour créer une nouvelle base de données ne contenant que certaines observations.

Exemple : on veut créer une table "junior" qui ne contient que les personnes de 12 ans et moins. 2 solutions sont possibles :

```
data junior ;  
set club ;  
if age <=12;  
run;
```

```
data junior;  
set club;  
if age>12 then delete;  
run;
```

On peut créer plusieurs bases dans la même étape DATA. Dans ce cas on déclare sur la ligne débutant par DATA le nom des bases que l'on souhaite créer. Puis on écrit les instructions conditionnelles permettant d'affecter les observations dans ces nouvelles bases, en fonction de conditions que l'on définit.

Exemple : on souhaite créer deux bases : la base « petits » et la base « grands » en fonction de la taille de l'enfant. Les observations pour lesquelles la taille est inférieure à 140 centimètres seront mises dans la base « petits », les autres seront mises dans la base « grands ».

```
data petits grands ;  
set club ;  
if taille <140 then output petits ;  
else output grands ;  
Run ;  
proc delete data=club;  
run;
```

Les fonctions conditionnelles et logiques (5/11)

Attention aux conditions imbriquées !

L'instruction [ELSE] ne porte que sur le IF immédiatement précédent :

Dans l'exemple suivant, l'instruction3 est vérifiée si l'instruction2 est non-vérifiée (que la condition1 soit vérifiée ou non)

```
IF condition1 THEN instruction1;  
IF condition2 THEN instruction2;  
ELSE instruction3 ;
```

Pour écrire le programme correctement, il faut écrire :

```
IF condition1 THEN instruction1;  
ELSE IF condition2 THEN instruction2 ;  
ELSE IF condition3 THEN instruction3 ;
```

Dans ce cas, l'instruction3 ne sera réalisée que si les conditions 1 et 2 ne sont pas vérifiées.

Les fonctions conditionnelles et logiques (6/11)

L'instruction conditionnelle **SELECT ... END ;**

Lorsqu'on a plusieurs instructions imbriquées il est préférable d'utiliser l'instruction **SELECT**. Cette instruction permet de sélectionner une variable, puis de dire pour chacune des modalités l'instruction qu'on veut lui associer. Cela simplifie la procédure et évite l'écriture de nombreuses lignes de programmation.

L'instruction commence par **SELECT** et se termine par **END** ;

Le mot clé **WHEN** introduit les modalités pour lesquelles on souhaite réaliser des instructions.

Lorsqu'une instruction **WHEN** est vérifiée, alors celles qui suivent ne le sont pas. **OTHERWISE** contient toutes les modalités non énumérées, tous les autres cas possibles. Il est souhaitable dans tous les cas d'utiliser **OTHERWISE** ; cela permet de rassembler tous les cas qu'on n'aurait pas vus au moment d'écrire la programmation.

Syntaxe :

```
data newbase ;  
set oldbase ;  
SELECT (variable) ;  
    WHEN (modalité1) instruction1;  
    WHEN (modalité2) instruction2;  
    OTHERWISE instruction3  
  
end;  
run;
```

Les fonctions conditionnelles et logiques (7/11)

Exemple : On souhaite créer une variable "prix" qui donnera le prix du tee-shirt du club. Ce prix est différent selon la taille de l'enfant. S'il mesure 130 ou 135 cm, le tee-shirt coûte 30, s'il mesure 145 le prix est de 40, s'il mesure 150 ou 155, le prix est de 50 et pour les autres tailles le prix est de 60.

```
data club2 ;  
  set club ;  
  select (taille) ;  
    when (130,135) prix=30 ;  
    when (145) prix=40 ;  
    when (150,155) prix=50 ;  
    otherwise prix=60 ;  
  End;  
Run;
```

Exemple : on souhaite créer une base « nageur » et une base « gymnaste » qui contiendront respectivement les enfants qui pratiquent la nage et ceux qui pratiquent la gym. On crée aussi une base « autres » pour le cas où des enfants ne pratiqueraient aucune de ces deux activités.

```
data nageur gymnaste autres ;  
  set club ;  
  select (activite) ;  
    when ('natation') output nageur ;  
    when ('gym') output gymnaste ;  
    otherwise output autres ;  
  end ;  
run;
```

Les fonctions conditionnelles et logiques (8/11)

Non spécification de variable

Dans les cas ci-dessus une variable de sélection a été spécifiée, mais il est possible de ne pas spécifier de variable au départ. La syntaxe sera alors :

```
Select ;  
  when (variable=modalité1) instruction1 ;  
  when (variable=modalite2) instruction2 ;  
  otherwise instruction3 ;  
End;
```

La non spécification est utile lorsque les conditions ne portent pas sur les même variables ou lorsque la condition porte sur plusieurs variables.

Réalisation de plusieurs instructions pour la même condition : Les blocs **DO ; ...END ;**

Si, pour une même condition, on souhaite que SAS effectue plusieurs instructions, ces instructions seront inscrites à l'intérieur d'un bloc « **DO** ». Ce bloc de programmation commence par le mot clé **DO ;** et se termine par **END ;**

Les blocs **DO ; ... END ;** sont utilisables aussi bien avec les instructions conditionnelles **IF... THEN ... ; ELSE....** qu'avec les instructions **SELECT... END ;**

Syntaxe avec l'instruction conditionnelle IF... THEN ... ; ELSE....

```
DATA newtable ; set oldtable ;  
IF condition then DO ;  
  Instruction1 ;  
  Instruction2 ;  
END ;  
ELSE DO ;  
  Instruction3 ;  
  Instruction4 ;  
END ;  
run ;
```

Les fonctions conditionnelles et logiques (9/11)

Exemple : on souhaite créer deux variables : "cotis" (le montant de la cotisation au club sportif) et "points" (le nombre de points de fidélité accordés). La cotisation et le nombre de points sont fonction de l'âge de l'individu. Pour les jeunes de plus de 12 ans, la cotisation est de 100 et on leur accorde 30 points, pour les autres la cotisation est de 80 et on leur accorde dix points.

```
Data club2 ; s et club ;  
IF age>12 then DO ;  
Cotis=100;  
Points=30;  
END ;  
ELSE DO ;  
Cotis=80 ;  
Points=10;  
END ;  
run ;
```

Syntaxe avec l'instruction conditionnelle **SELECT... END ;**

Ne pas oublier que chaque instruction **SELECT** se termine par un **END** ; ET que chaque bloc **DO** ; se termine également par un **END** ;.

```
data newtable ;  
set oldtable ;  
SELECT (variable);  
WHEN (modalité1) DO ;  
instruction1 ;  
instruction2 ;  
END ;  
WHEN (modalité2) instruction3;  
OTHERWISE instruction4 ;  
END ;  
run ;
```

Les fonctions conditionnelles et logiques (10/11)

L'instruction WHERE:

Contrairement à l'instruction *IF* qui ne peut s'employer que dans une étape DATA, l'instruction WHERE s'utilise aussi bien dans une étape DATA que dans une étape *PROC*.

La syntaxe de l'instruction *WHERE* est la suivante : *where condition* ;

Contrairement à l'instruction IF dans laquelle SAS parcourt chaque ligne de la base pour sélectionner les observations demandées, l'instruction where sélectionne dès la lecture de la table les observations remplissant les conditions. Lorsqu'on exploite de grosses bases de données cela permet un gain de temps lors de la réalisation de l'instruction.

Exemples

```
proc freq data=club ;  
  where sexe='H' and activite='natation';  
  table age ;  
run ;
```

Dans l'exemple précédent, on effectue un dénombrement de la variable "age" mais grâce à l'instruction *where* on a au préalable filtré la population, en ne conservant que les hommes qui pratiquent la natation. L'output donnera donc la distribution de l'âge pour les nageurs hommes (et seulement eux).

```
Data junior ;  
set club ;  
where age<12 ;  
run;
```

Dans cet exemple, on crée une base "junior" qui ne contient que les enfants ayant strictement moins de 12 ans.

Remarque : ici, « *where age<12* » est équivalent à « *if age<12* »

Les fonctions conditionnelles et logiques (11/11)

Remarques

- Tous les opérateurs vus avec l'instruction **IF** restent valables.
- L'instruction **WHERE** ne peut pas porter sur une variable calculée
- Il ne peut y avoir qu'un seul « **WHERE** » par étape DATA
- **Where** n'est suivi d'aucune instruction (where... then... N'EXISTE PAS)

Particularité : "where" mis en option

L'instruction « where » peut être utilisée comme une option de la table. Cela permet un gain de temps lors de l'exploitation de bases de données importantes.

La syntaxe générale sera la suivante : **(where = (condition))** ;

Cette instruction peut s'utiliser au moment de l'écriture de la nouvelle table, c'est-à-dire sur la ligne commençant par le mot clé DATA ou au moment de la lecture de la table initiale, c'est-à-dire sur la ligne débutant par **SET**.

Exemples :

```
data hommes ;  
set club (where=(sexe='H')) ;  
run;
```